

Build optimization

Module 6 - Lesson 2

Overview

A fast build produces a fast site. Splitting code, compressing assets, and trimming dependencies shrink what users download and how long the build takes.

Key Concepts

- Code-split large routes with `dynamic import()` so users only load what they need.
- Bundle analyzers (`vite-bundle-visualizer`) reveal the largest dependencies.
- Compress images and serve them in modern formats (webp/avif) at the right sizes.
- Tree-shaking removes unused exports when imports are kept pure.
- Cache assets with content hashes so browsers reuse unchanged files.
- Preload critical assets and defer non-critical scripts.

Example

```
// dynamic import for a heavy route
const Reports = lazy(() => import('./Reports'));

// or in Next.js
const Reports = dynamic(() => import('./Reports'), {
  loading: () => <Spinner />,
});
```

Summary

Build optimization is about shipping less and caching well. Code-splitting, image compression, tree-shaking, and hashed assets make pages load faster on slow connections.

Practice Checklist

- Split your heaviest route with a dynamic import and measure the chunk sizes.
- Run a bundle analyzer and replace or remove the largest unnecessary dependency.
- Compress hero images and verify format and dimensions in the network tab.